
[Home](#) / [Resources — Blog](#) / 8 Simple Ways To Become Better at JavaScript!

[JavaScript](#)[Frontend](#)

8 Simple Ways To Become Better at JavaScript!

Walk through 8 simple, yet extremely practical, ways of improving your JavaScript skills.

27 mins · Thu Oct 10 2024



[Skip to main content](#)

Disclosure: The following content includes affiliate links. If you buy something through these links, I'll earn a commission without any extra cost to you. Your support is appreciated!

Introduction

Without a doubt, JavaScript is one of the most popular programming languages of all time, by virtue of its core usage on the client side — it's the only scripting language that browsers understand — and also on the server side since the inception of Node.js.

Learning JavaScript is an extremely lucrative thing to get into as it allows us to explore many different arenas in computer science, the likes of web development, app development, game development, data science, and so on.

While it's surely a great language to pick for getting into programming, JavaScript isn't a minimal language comprised of just a few interfaces to explore. Not at all! It is one beast of a language. The bits and pieces of the interfaces are simple to understand but overall the language is flooding with shiny bells and whistles.

It's not that challenging to become fluent in JavaScript, but equally it isn't that easy either.

However, in this article, we cover 8 ways for you to get better, better, and better than ever in JavaScript. Ways to become confident with your knowledge of JavaScript. Ways for you to be able to

[Skip to main content](#)

develop complex applications with ease.

Ways for you to teach JavaScript's complex parts to your friends. Ways to excel as a JavaScript dev.

So shall we get into them?

Pick a good JavaScript course

Needless to say, if you opt for a resource that's crafted poorly with a vague structure, pitching in ad-hoc ideas for you to learn, you'll never ever build that confidence to code in JavaScript. You'll always find yourself stuck in what's commonly referred to as a *'tutorial hell'*.

You'll be consuming tutorial after tutorial, resource after resource, but still not able to solve very basic exercises when the time comes to finally execute the learned concepts.

The last thing you want in a course, teaching something as large and complex as JavaScript, is a quick skim-through collection of already-cooked recipes to build feature A or B. *That's just not going to work! Ever!*

[Skip to main content](#)

Ideally, you want a course that doesn't just contain precooked recipes but instead also contains all the necessary learning materials for you to build your foundation and be able to cook anything yourself, whichever way you like. Even if there are precooked recipes, the intuition behind them must be made clear.

Now while finding a good learning resource for JavaScript is easy, **finding a bad one is easier**. There are tons and tons of bad resources out there. (We're sure you know of them!) A beginner might fall for the extensive lengths of content covered on website A or B without being able to judge that all that content is merely an extension of one single topic.

So how do you find a good resource for learning JavaScript?

Well, at least for JavaScript you don't need to worry that much, for we've got you covered.

At Codeguage, we've compiled one of the most comprehensive JavaScript courses out there, iterating on the course structure numerous times to make sure that it is easy to follow through and valuable for beginners as well as expert developers.

[Skip to main content](#)

There are superbly comprehensive chapters covering all kinds of different aspects of the

language, numerous exercises for you to practice the concepts that you learn, and a bunch of quizzes to test your knowledge in a quantitative way.

And if that's not enough, we're even working on building one of the largest practice question collections of JavaScript online, currently standing at a tally of 309 questions and covering JavaScript's foundation, numbers, and strings. (More questions will follow up in the coming days.)

Start reading books

Besides learning via online courses, we can't emphasize any more on the sheer importance of learning via books. Yes, *books!*

Do you love reading books in general? No? Well, at the very least, you should start to love reading books on JavaScript.

It won't be wrong to assume that well over a thousand books have been published on JavaScript since the day of its inception. JavaScript is everywhere, so how can that ever be true without it being in one of the most supreme resources of all times — books.

[Skip to main content](#)

Books carry immense value, and are typically written by experts in their fields, with rigorous editorial checks applied, again by industry experts, to make sure that the content covered is of exceptional quality.

So if you aspire to become a proficient JavaScript developer, start reading books starting from today!

Now you might go on and say *"Well, you know what, books are somewhat expensive and add to my minimal desk setup. So maybe I am better off without them when learning JavaScript."*

Now while you're not wrong in your first proposition, you are definitely wrong in your second one.

We beg to argue on the fact that how can anyone be better off without reading books?

Believe it or not, even reading one (good) book on JavaScript can bring about a great change in your development skills, especially if that book is based on building a project or multiple projects rather than explaining syntax and language theory using ad-hoc examples. The latter kind is also really helpful, most importantly as a source of quick reference, but in the longer run, you'll value books where authors build

[Skip to main content](#)

an app and along with that explain different concepts.

Anyways, coming back to your statement, yes **books are expensive** and **add to one's physical space** but there are great solutions to these problems in today's digital era.

One of our favorites is **Perlego**. For just **\$8/month**, you get to browse from a humongous directory of books, from notable publishers such as Packt, Manning, and Apress, in addition to many many others.

Here are three amazing books that we recommend everyone to read for improving their JavaScript skills:

- *Professional JavaScript for Web Developers*, Matt Frisbie
- *Testing JavaScript Applications*, Lucas Fernandes da Costa
- *JavaScript All-in-One For Dummies*, Chris Minnick

If you can go a bit higher than \$8, then we'd recommend you to definitely also try out subscribing to **O'Reilly**.

Skip to main content

You might've heard of O'Reilly before, but even if you haven't, no problem. Now is the high time to know about it.

O'Reilly is a book publisher, producing an elite collection of books on computers and technology. Seriously, a book from O'Reilly is one big testimonial that it's not just good, but filled with value dripping in every single sentence.

For just **\$49/month**, you get to browse from a jaw-dropping inventory of exceptional-quality books, covering different aspects of learning JavaScript.

Some great books worth learning from O'Reilly on JavaScript include:

- *JavaScript: The Definitive Guide, 7th Edition*, David Flanagan
- *Learning JavaScript Design Patterns, 2nd Edition*, Addy Osmani

The biggest benefit of purchasing the subscription of O'Reilly is that in addition to books from O'Reilly, you get access to books from other notable publishers as well, including Packt, No Starch Press, Manning, Wiley, Addison-Wesley Professional, etc.

[Skip to main content](#)

Get your HTML DOM knowledge right

For frontend developers, DOM mutations are something they do day in, day out. *Nothing denying that, right?*

Every JavaScript app development framework/library, including the well-known React, Vue and Angular, abstracts away the



Practice ▾ The DOM interface is *superbly huge*. ▾ Projects ▾ Pricing ▾ Resources ▾ Community ▾

Well that got lost. Let's restate it. The DOM interface is *superbly huge*. Lost again, maybe. Let's do it once more. The DOM interface in JavaScript is *superbly huge*. There is honestly a heck lot of stuff to cover in it.

Sign in

Sign up

Note: We're considering both the CSSOM and the Events API in the DOM here, even though precisely speaking, they are different ideas.

Knowledge of the DOM is superbly important for every JavaScript developer. Almost every interaction conducted by the user in the browser on any website ends up utilizing some kind of a feature from the DOM.

Skip to main content

Therefore, you must make sure to solidify your comprehension of the DOM APIs, and be able to answer such questions as:

- What is the `Node` interface?
- What is the `Element` interface?
- *How to add a new element before a given element?*
- *How to add a new element after a given element?*
- What are document fragments and when might we need them?

...and so on and so forth.

In our JavaScript course, we have whole units dedicated to exploring the HTML DOM, the CSSOM and then the Events API in the language. There are numerous exercises for you to tackle to build the necessary fluency of working with the DOM.

A quick exercise for you

As one teaser of an exercise from the course, try creating a `getInnerHTML()` function that replicates the behavior of `innerHTML` when we get it.

That is, given a `Node` instance `node`, `getInnerHTML(node)` should return

[Skip to main content](#)

the same thing as `node.innerHTML` .

The main point of the exercise is to **NOT** use `innerHTML` , otherwise there's clearly no point of the exercise!

The full exercise can be read at [Exercise: JavaScript HTML DOM — HTML Serialization.](#)

Get familiar with the design of JavaScript

At some point in time, becoming better at any given programming language comes down to digging into its internals and trying to make some sense from them on how the language works.

For JavaScript, to name a few, you should know about its asynchronous, event-driven execution model; about its prototypal object-oriented nature; and about its strange typing quirks.

Going a bit more specific, you should know such things as:

Skip to main content

- [*What is the event loop and how it works?*](#)

- *What are microtasks?*
- *Why does the second argument to `setTimeout()` specify the minimum time span after which the given callback fires and not the exact time span?*
- *Why are classes in JavaScript not the same as classes from classic OOP languages, such as Java?*

...and so on and so forth.

But these are questions that don't necessarily require one to dig into the very internals of any implementation of JavaScript; they can, more or less, be answered by theoretical discussions, keeping concrete engine details away.

However, there are certain questions that can only be answered by considering some insights into engine constructions.

These include:

- *How to create objects with the same 'shape' for the sake of better performance?*
- *How to make sure garbage collection doesn't happen often (which can otherwise lead to subpar performance)?*

Skip to main content

- *Why keep from assigning arbitrary indexes to arrays in JavaScript in order to prevent them from becoming sparse arrays?*

Perhaps the best resource to learn about these internals is from the same team behind the best tool that implements them, **V8**. V8 is Google's open-source JavaScript engine, using a JIT-compiling mechanism to execute a JavaScript program.

The blog of V8 has some amazing resources worth reading to get some invaluable insights into JavaScript.

Besides learning from V8's blog, two other blogs worth considering are as follows:

- Mathias Bynens
- Benedikt Meurer (Here, search for the 'javascript' tag from the side menu and skim through the collection of posts.)

Clearly, learning this deeper knowledge first requires you to be comfortable with the fundamentals of JavaScript. And that comes with a good resource, some solid amount of time, and some great amount of practice.

Skip to main content

Listen to interviews and conferences

"Interviews? Conferences? How could they possibly help me learn JavaScript?", you ask. Well, in every way!

An interview — such as the one between Lex Fridman and the creator of JavaScript, Brendan Eich — and a conference talk — such as the one that Ryan Dahl gave on the birth of Node.js — are active mediums of learning in contrast to static, passive documents teaching us given concepts via text.

In interviews and conferences, you get to see different tangents of people on different ideas, with the presenters usually talking about numerous side things besides what they are actually presenting.

When you listen to talks revolving around JavaScript, you get to harness a great deal of knowledge from these active conversations and know about various aspects of the language.

So where do you get started? Where to begin watching these talks, interviews, and conferences?

[Skip to main content](#)

Well, it's not really difficult. Head over to YouTube (or even Google, but that won't produce very good results in this case) and search for '*JavaScript conferences*'. Skim through the videos that appear and shortlist the ones whose topics align with your current interests.

And pretty soon, you'll get similar suggestions in your YouTube feed once you watch either of these videos, giving you a steady stream of talks to listen to.

Obviously, this is going to take some effort on your end to find a talk/conference/interview whose topic is watch-worthy, but believe it, the result is worth it all.

Continuously perform simple exercises

Regularly solving simple JavaScript exercises is just like going to the gym daily and lifting small weights — it keeps you in shape.

[Skip to main content](#)

And if you tackle a bit harder ones, similar to lifting slightly heavier weights which ultimately end up with you having bigger

and stronger muscles, you get to build some exceptional programming and problem-solving skills.

Easy formula, isn't it?

So if you aspire to become a pro JavaScript developer, it's impossible to do so without regularly solving basic, simple computational problems using it.

So does this mean I should implement a basic prompt-input-based calculator in JavaScript? Yes.

Does this mean I should implement a basic integer check in JavaScript? Absolutely yes.

Does this mean I should write a function to find the maximum number in a given array of numbers? Indeed, yes.

The thing is that even though some exercises might seem so elementary to solve that we'd feel tempted to skip them, it's these elementary exercises that help us build muscle memory for typing given syntactic elements of a language (symbols, keywords, indentations) quickly and also keep our problem-solving hygiene checked.

[Skip to main content](#)

These simple exercises help us get a quick warm-up before approaching to solve more

difficult programming problems in the near future.

They also get us to start building and following kind-of our own system of coding conventions, for e.g. possibly using the Allman style or maybe the K&R style for denoting curly braces in code; having `else` on the same line as the ending `}` of the previous `if` ; and so on.

These exercises help us remember the utility of different language features of JavaScript, which in turn pays us off when we go on to build something really complicated.

Surely, the last thing you want while coding an application in JavaScript is not knowing how to determine if an arbitrary value is an integer or not.

Develop an analytical eye

The next time you visit a website, don't just skim through it and then exit it. No. Try to see the website from an analytical eye, judging different functionalities on the

[Skip to main content](#)

website and trying to reverse engineer them.

For instance, let's say you go to a website to read a blog post. Now, by virtue of your analytical eye, you notice a search bar at the top reading something along the lines *"Press '/' to start searching."*

Immediately this piques your curiosity and leaves you wondering about how this functionality might have been implemented.

But you're not there to just wonder, are you?

You go on, spin up a dummy webpage, dump in an `<input>` element in it, and then try cracking setting up a similar functionality in this webpage.

You think about the event required to tap into listening to keyboard events throughout the entire document. Right away, the keydown event rings bell. *keydown on what?* `window`, definitely.

But how to check for the `/` key? Using the `keyCode` property, or even the more modern `key` property, on the given `KeyboardEvent` object provided to the `keydown` handler.

Skip to main content

You get it, right?

Do give a try to processing websites in this way the next time you see them, thinking at the back of your mind how different functionalities might have been implemented.

Sometimes, such treks can lead you long into territory that you haven't ever explored before. Who knows, maybe such a trek gets you to ultimately learn about the Selection API, or the Clipboard API, or maybe the Drag and Drop API, and so on.

Ask questions when you get stuck

Reading the heading here, before you go on and ask literally every single question that pops in your mind on StackOverflow or other questionnaire websites, hear this out:

"Keep yourself from asking superbly fundamental questions that you should be knowing from whatever resource you're learning JavaScript in the first place."

[Skip to main content](#)

If you don't know answers to fundamental questions, try another resource for learning JavaScript.

Try **NOT** to ask questions like:

- *How to check whether an arbitrary value is an integer?*
- *How to find a given element's index in an array?*
- *Why does '😊'.length return 2 instead of 1 ?*
- *How to check for a NaN value in JavaScript?*

These questions aren't really subjective — they are purely based on the design and nature of JavaScript, and henceforth, shouldn't end up as new questions of the future. Plus, they've already been answered so there's even less point in asking them once again.

Ask deeper questions or questions that are, more or less, subjective.

For instance, one possible question (although we aren't really setting up a benchmark here, nor stating that such a question hasn't already been answered) might be "*What's the point of the Math.imul() method in JavaScript?*"

[Skip to main content](#)

Keep in mind that we aren't strictly warning you against asking questions that are close to fundamental; we're just trying to say that you should keep yourself from initiating such questions as much as you can, for their answers are usually objective, not subjective, only requiring you to put in some effort and get your fundamentals intact.

Subjective questions usually get one to learn a lot.

Here's one possible scenario where asking a question might help.

Creating an application and have two different approaches to take to implement a feature A? Can't decide which approach to take probably because you're not that experienced in building such a feature?

Well, this is the perfect point to ask for help from an expert.

Questionnaire sites such as Reddit and StackOverflow host a great number of experts who can help you out with your problem, and in that way, help you gain a considerable amount of knowledge regarding JavaScript.

[Skip to main content](#)

Wrapping up

And that's it guys for our 8 simple ways to become better at JavaScript.

Honestly speaking, mastering JavaScript isn't *as hard* as people out there tend to portray it. After all, teams are developing such complex applications as Figma, using extremely low-level JavaScript interfaces, including those allowing us to work with memory buffers, OpenGL, audio, and much more.

The complexity of JavaScript apps is on the rise and with Node.js, enabling us to hop on to exploring the server end as well, it's only going to become even more prevalent in the near future.

So without wasting any more of your time, put these 8 ways into action starting from now! Or if you've already gone through either one of them already before, share your experience with us by tagging us (@codeguage) on Twitter.

Keep learning and keep growing! And please take a small break from your screen, for this article was quite a long one. 😊



Skip to main content

Become confident with JavaScript, not frustrated

Work your way through a large collection of practice material and put your knowledge into real action.

Start practicing



Exercise 1 - Very easy

[Foundation - Addition Calculator Exercise](#)

Exercise 2 - Very easy

[Foundation - Simple Swapping Exercise](#)

Exercise 3 - Very easy

[Foundation - Sorted Numbers Exercise](#)

Related articles

Think You Know the DOM? Prove It With These 10 Exercises!

11 mins · Wed Aug 20 2025



[Skip to main content](#)

You Know JavaScript But Can't Solve Problems in It?

16 mins ▪ Wed Aug 20 2025



JavaScript Interview Question: Compute the Sum of a List of Numbers, Recursively

6 mins ▪ Wed Aug 13 2025



[View all articles →](#)

Content

[Practiceables](#)

[Projects](#)

Practice

[Quizzes](#)

[Exercises](#)

[Questions](#)

Resources

[Blog](#)

[Glossary](#)

[Books to read](#)

[Courses](#)

Deprecated

[Pricing](#)

[Announcements](#)

[Skip to main content](#)

[About](#)

[Connect](#)

[Contact](#)

[FAQs](#)

[About](#)

[Codeguage](#)

[My story](#)



For any queries or
suggestions, write to me at
bilal@codeguage.com



Codeguage

[Affiliate disclosure](#)

[Terms of use](#)

[Privacy policy](#)

Codeguage © 2026. All rights reserved. Made with ❤️ by [Bilal Adnan](#).